

ADD THESE TWO FEATURES TO THE EXISTING ARTHAYOG DORMITORY APPLICATION:

IMPORTANT:

The application is already deployed. Do NOT rebuild the application, create a new project, replace the existing architecture, or remove working features. First inspect the existing frontend, backend, database, authentication, booking, payment, and reusable components. Implement these features incrementally using the existing system and real database data.

=====

1. BOOKING CALENDAR

=====

Add a "Booking Calendar" section inside the existing Owner/Staff booking area.

The calendar must display REAL bookings from the existing database. Do NOT use fake, random, or hardcoded bookings.

The calendar should support:

- Monthly view
- Weekly view where practical
- Date navigation
- Selecting/clicking a booking to view its complete details

Each booking shown on the calendar should clearly display relevant information such as:

- Customer or group name
- Booking ID
- Check-in date
- Check-out date
- Floor
- Number of beds
- Booking type
- Booking status
- Payment status

For example, a wedding/group booking could appear as:

Wedding Group
Floor 1 + Floor 2
11 Beds
15 Oct – 17 Oct
Confirmed

For an individual booking:

Rahul Patil

Floor 3
2 Beds
15 Oct – 16 Oct
Confirmed

For group/wedding bookings, clearly show the selected floor(s) and total number of assigned beds.

The calendar must be connected to the existing booking database.

When a booking is:

- Created
- Modified
- Cancelled
- Expired
- Checked in
- Checked out

the calendar must automatically reflect the updated status.

Clicking a calendar booking should open the existing booking details page/modal rather than creating a separate duplicate booking system.

The calendar must respect the existing date-based availability and booking logic.

Do NOT allow the calendar to create double bookings.

The calendar should be useful for quickly understanding:

- Today's bookings
- Upcoming check-ins
- Upcoming check-outs
- Group/wedding bookings
- Occupied/reserved periods

Keep the design simple and professional. Do not add unnecessary complicated calendar functionality.

=====
2. INVOICE / RECEIPT
=====

Add a professional booking Invoice/Receipt feature.

After a booking is successfully confirmed and the applicable payment has been verified, provide:

- View Receipt

- Download Receipt
- Print Receipt

Where practical, allow the receipt to be accessed again from the booking details/history.

The receipt must use REAL booking and payment information from the database.

Include:

PROPERTY INFORMATION:

- Arthayog Dormitory name
- Logo if configured
- Phone
- WhatsApp where configured
- Email where configured
- Address
- Other owner-configured property information

CUSTOMER INFORMATION:

- Customer name
- Customer phone
- Customer email where available

BOOKING INFORMATION:

- Booking ID
- Booking creation date
- Check-in date
- Check-out date
- Check-in time
- Check-out time
- Booking type
- Individual or Group/Wedding Booking
- Floor(s)
- Assigned bed(s) or total number of beds

PAYMENT INFORMATION:

- Price per bed where applicable
- Number of beds
- Total booking amount
- Amount paid
- Payment status
- Payment/transaction reference where available from the verified payment system
- Payment date where available

BOOKING STATUS:

- Confirmed

- Checked-in
- Checked-out
- Cancelled, where appropriate

For a group/wedding booking, show information clearly, for example:

Group Booking — Wedding
 Floor 1 + Floor 2
 11 Beds
 Total Amount: ₹XXXX
 Payment Status: SUCCESS

IMPORTANT PRICE RULE:

The receipt must use the exact price stored with the confirmed booking.

If the owner later changes the current price, old receipts and confirmed bookings must NOT change.

Example:

Booking confirmed at ₹500.
 Owner later changes current price to ₹600.
 The old booking receipt must still show ₹500.

IMPORTANT PAYMENT RULE:

Never generate a receipt that falsely says payment was successful.

Only show SUCCESS/PAID when the existing backend payment verification confirms the payment.

If payment is pending or failed, clearly show the correct payment status instead.

Do NOT create fake payment IDs or transaction IDs.

=====

3. RECEIPT DESIGN

=====

Make the receipt professional, clean, and suitable for a real small dormitory.

Use the existing Arthayog visual style:

- Warm white
- White
- Dark text
- Soft borders
- Muted earthy/terracotta accent
- Clean spacing

Do not make it look like a generic ERP invoice.

It should be easy to read on:

- Mobile
- Tablet
- Desktop
- Printed paper

Make the receipt mobile-friendly.

If a PDF/download implementation already exists, integrate with it rather than creating an unnecessary separate system.

=====

4. GROUP/WEDDING BOOKINGS

=====

The new calendar and receipt must fully support the existing Group/Wedding Booking functionality.

For example:

Floor 1 = 5 beds

Floor 2 = 6 beds

A wedding booking can select:

Floor 1 + Floor 2

The system automatically assigns:

11 beds

The Booking Calendar must show:

Wedding Group

Floor 1 + Floor 2

11 Beds

The Receipt must show:

Group/Wedding Booking

Floor 1 + Floor 2

11 Beds

Do not require staff to manually enter the same information again.

Use the existing booking → selected floors → assigned beds relationship.

=====

5. OWNER / STAFF ACCESS

=====

Owner and authorized Staff should be able to:

- View the Booking Calendar
- Open booking details
- View receipts
- Download receipts
- Print receipts

Customers should be able to:

- View their own confirmed booking
- View their own receipt
- Download/print their own receipt where appropriate

Customers must never be able to access another customer's booking or receipt.

Enforce this through backend authorization, not just frontend hiding.

=====

6. DATABASE AND EXISTING ARCHITECTURE

=====

Use the existing database and booking/payment architecture.

Do NOT create duplicate booking tables or a separate calendar database.

The calendar is only a visual representation of existing bookings.

The receipt is generated from existing booking, customer, property, price, and verified payment data.

Expected flow:

Existing Booking

→ Database

→ Booking Calendar

Existing Confirmed Booking + Verified Payment

→ Database

→ Invoice/Receipt

→ View / Download / Print

Do not change existing authentication, payment, or booking architecture unless strictly necessary.

=====

7. REAL DATA ONLY

=====

STRICT REQUIREMENT:

Do not create:

- Fake bookings
- Fake customers
- Fake prices
- Fake payment IDs
- Fake transaction IDs
- Fake revenue
- Fake calendar entries
- Fake receipts

Everything must come from the actual database.

If test data is required during development, keep it completely separate from production data.

=====

8. FINAL TESTING

=====

After implementation, test:

BOOKING CALENDAR:

1. Create individual booking → appears on calendar.
2. Create wedding/group booking → appears on calendar.
3. Book Floor 1 + Floor 2 → calendar shows 11 beds.
4. Modify booking → calendar updates.
5. Cancel booking → calendar updates.
6. Check-in → calendar reflects status.
7. Check-out → calendar reflects status.
8. Change booking dates → calendar updates.
9. Attempt overlapping booking → existing protection must prevent double booking.

RECEIPT:

1. Confirm normal booking → receipt generated.
2. Confirm group booking → receipt shows selected floors and total beds.
3. Verify payment → receipt shows correct verified payment status.
4. Payment failure → receipt must not falsely show PAID.
5. Change current owner price after booking → old receipt remains unchanged.
6. Download receipt → correct data.
7. Print receipt → clean printable layout.
8. Customer can only access their own receipt.
9. Owner/authorized staff can access appropriate receipts.

=====

FINAL INSTRUCTION

=====

Add ONLY these two improvements:

1. BOOKING CALENDAR
2. INVOICE / RECEIPT

Do not add unnecessary new features.

Do not rebuild the application.

Do not delete existing data.

Do not replace working functionality.

Use the existing frontend, backend, database, authentication, booking system, payment verification, group booking system, and reusable components.

The final workflow should be:

Availability

- Booking
- Individual or Group/Wedding Booking
- Payment
- Confirmation
- Booking Calendar
- Check-in
- Check-out
- Invoice/Receipt
- Reports

Keep everything simple, professional, reliable, and suitable for the real operation of the 16-bed Arthayog Dormitory.

Improve the existing multilingual/i18n system without rebuilding the application.

The current English → Marathi/Hindi translation is too literal and sometimes changes the intended meaning. Fix this by using context-aware, meaning-based localization.

Requirements:

1. Keep English, Marathi (मराठी), and Hindi (हिंदी).
2. Do NOT translate UI text word-for-word. Each translation must preserve the intended meaning, context, tone, and purpose of the original English text.

3. Marathi and Hindi should sound natural to native Indian users, especially for a professional dormitory/accommodation booking website.

4. Create/use structured translation keys instead of dynamically translating English strings at runtime.

Example:

booking.bookStay

English: Book Your Stay

Marathi: आपला मुक्काम आरक्षित करा

Hindi: अपना ठहराव बुक करें

booking.confirm

English: Confirm Booking

Marathi: आरक्षण निश्चित करा

Hindi: बुकिंग की पुष्टि करें

5. Review and improve all important customer-facing translations including:

- Navigation
- Home/About
- Live Availability
- Booking
- Check-in
- Check-out
- Bed availability
- Pricing
- Payment
- Booking confirmation
- My Bookings
- Receipt/Invoice
- Contact
- Facilities/Amenities
- FAQ
- Buttons
- Form labels
- Validation messages
- Error messages
- Success messages
- Cancellation messages

6. Use hospitality/accommodation terminology that sounds natural in Marathi and Hindi rather than literal dictionary translations.

7. Preserve the intended meaning. If a direct translation sounds unnatural, use the natural equivalent instead.

8. Do not translate:

- Customer names
- Phone numbers
- Email addresses
- Booking IDs
- Payment IDs
- User-entered addresses
- User-entered special instructions

9. Owner-configurable customer-facing content should support separate English, Marathi, and Hindi values where appropriate instead of automatically translating one language into another.

10. Add a fallback:

If a Marathi/Hindi translation is missing, show the configured default language text instead of showing translation keys such as booking.confirm or broken placeholders.

11. Do not lose form data when the user changes language.

12. Ensure Marathi and Hindi Unicode characters render correctly on mobile, tablet, and desktop.

13. Review the complete existing translation files and existing UI before making changes. Do not create a separate translation system if one already exists. Improve the current implementation.

14. Do not change authentication, booking logic, payment logic, database structure, or other working features unnecessarily.

15. After implementation, test the same complete booking flow in:

English → Marathi → Hindi

Check that the meaning, buttons, validation messages, payment text, confirmation text, and booking information remain correct in all three languages.

The final result should feel like the application was genuinely written for Marathi/Hindi users, not like English text was mechanically translated.

Improve the existing "Daily Operations" / "Front Desk Duty Station" page.

Do NOT create a new page and do NOT rebuild the existing system.

The current Daily Operations page duplicates functionality that already exists in the top navigation. It currently shows shortcuts such as Live Beds, Bookings Ledger, Supplies Inventory, and Group/Wedding Booking, which makes it feel like another navigation menu.

Change its purpose:

Daily Operations must become the actual FRONT DESK DAILY DASHBOARD, answering:
"What does the staff member need to handle today?"

Keep the existing top navigation:

- Daily Operations
- 16 Beds
- Bookings
- Inventory
- Language
- Phone Booking

Do not duplicate these navigation sections as large cards on Daily Operations.

Instead, redesign Daily Operations around today's operational work.

Show a compact summary at the top:

- Ready Beds
- Occupied Beds
- Today's Check-ins
- Today's Check-outs
- Pending Bookings / Pending Payments where applicable

All numbers MUST come from the real database and current date. Never use fake, random, or hardcoded production data.

Then show:

1. TODAY'S CHECK-INS

Display actual bookings arriving today:

- Guest/customer or group name
- Booking ID/reference
- Floor and assigned bed(s)
- Booking type
- Payment status
- Booking status
- Check-in time if configured
- Check In action

2. TODAY'S CHECK-OUTS

Display actual bookings leaving today:

- Guest/customer or group name
- Booking ID/reference
- Floor and assigned bed(s)
- Check-out time
- Check-out action

3. PENDING ACTIONS

Only show actions that actually require staff attention, for example:

- Pending payment
- Booking awaiting confirmation
- Booking requiring verification
- Beds that need to be marked ready after checkout

Do not invent actions if there are none. Show a clean "No pending actions" state.

4. QUICK ACTIONS

Keep only genuinely useful front-desk actions:

- Phone Booking
- Group/Wedding Booking
- View Live Beds

These should be quick actions, not duplicated navigation sections.

Do NOT make separate large cards for:

- Bookings Ledger
- Inventory
- Supplies Inventory
- Live Beds

because those already exist in the main navigation.

5. Keep the existing clean Arthayog visual style:

- warm white background
- professional hospitality look
- compact spacing
- clear hierarchy
- moderate text sizes
- minimal unnecessary decoration
- mobile/tablet/desktop responsive

6. Important:

The screenshot currently shows values such as "16 Ready Beds" and "27 Today's Check-ins". Verify the logic behind these numbers. The dashboard must use real database calculations and current-date filtering. Do not display fabricated/test values in the production interface.

7. Preserve all existing booking, bed, payment, authentication, role authorization, inventory, and database functionality.

The final Daily Operations page should feel like a FRONT DESK WORKSPACE, not another menu.

Desired flow:

Staff logs in

→ Daily Operations

→ sees today's arrivals/departures/pending actions

→ performs check-in/check-out/payment/booking actions

→ uses top navigation only when they need the full Beds, Bookings, or Inventory sections.

Update the existing Arthayog Dormitory application incrementally.

IMPORTANT:

- Do NOT rebuild the application.
- Do NOT create a new application.
- Do NOT delete or reset the existing database.
- Do NOT remove working features.
- First inspect the current frontend, backend, database, authentication, booking, payment, availability, and role-authorization implementation.
- Integrate the changes into the existing system.
- Use real database/configuration data only.
- Do not create fake, random, or hardcoded production data.

=====

1. REDESIGN "DAILY OPERATIONS" AS A REAL FRONT DESK DASHBOARD

=====

The current "Daily Operations / Front Desk Duty Station" page duplicates sections that already exist in the top navigation.

It currently contains shortcuts such as:

- Live Beds
- Bookings
- Inventory
- Group/Wedding Booking
- Supplies Inventory

These should NOT appear as large duplicate navigation cards.

The purpose of Daily Operations should be:

"What does the staff member need to handle TODAY?"

Keep the existing top navigation:

- Daily Operations
- 16 Beds
- Bookings
- Inventory
- Language
- Phone Booking

Daily Operations should become the staff's daily operational workspace.

TOP SUMMARY

Show compact real-time statistics:

- Ready Beds
- Occupied Beds
- Today's Check-ins
- Today's Check-outs
- Pending Booking Requests
- Pending Payments where applicable

All values MUST come from the real database and current date.

Never use fake, random, or hardcoded numbers.

For a 16-bed property, ensure the calculations are logically consistent with actual bookings and bed states.

TODAY'S CHECK-INS

Show actual bookings arriving today.

For each booking show:

- Customer/Group name
- Booking ID
- Check-in date
- Floor
- Assigned bed(s)
- Booking type
- Payment status
- Booking status
- Check-in time if configured

Provide the appropriate:

[Check In]

action.

TODAY'S CHECK-OUTS

Show actual bookings leaving today.

Display:

- Customer/Group name
- Booking ID
- Floor
- Assigned bed(s)
- Check-out date/time
- Booking status

Provide:

[Check Out]

action.

After checkout, update the bed availability/readiness using the existing database logic.

PENDING ACTIONS

Show only actions that actually require staff attention.

Examples:

- Booking awaiting approval
- Payment awaiting customer payment
- Payment requiring verification
- Booking requiring staff attention
- Beds that need to become ready after checkout

If there are no pending actions, show:

"No pending actions"

Do not create fake actions.

QUICK ACTIONS

Keep only useful front-desk actions:

- Phone Booking
- Group/Wedding Booking
- View Live Beds

Do NOT create duplicate large cards for:

- Bookings
- Inventory
- Supplies Inventory
- Live Beds

Those already exist in the main navigation.

Daily Operations should feel like a FRONT DESK WORKSPACE, not another navigation menu.

=====

2. REMOVE WHATSAPP — CALL ONLY

=====

Remove WhatsApp completely from the application.

Do not show:

- WhatsApp buttons
- WhatsApp icons
- WhatsApp contact actions
- "Contact on WhatsApp"
- WhatsApp links

Keep only CALL as the direct contact option.

OWNER-CONFIGURABLE PHONE NUMBERS

The Owner must be able to configure multiple phone numbers.

Add/use:

Owner Settings
→ Contact Settings
→ Phone Numbers

Owner can:

- Add phone number
- Edit phone number
- Remove phone number
- Set a display name/label
- Set active/inactive
- Control which numbers are shown to customers

Example labels:

- Front Desk
- Owner
- Day Staff
- Night Staff

These are examples only.

Do NOT hardcode actual phone numbers.

Customer-facing pages should show all active owner-configured phone numbers.

Each number should have a CALL action that uses the device's normal telephone action.

Only active configured numbers should appear.

=====

3. CHANGE CUSTOMER SELF-BOOKING TO REQUIRE APPROVAL

=====

Currently a customer can select a booking and proceed directly to payment.

Change this workflow.

The customer must FIRST submit a booking request.

The booking must then be approved by an authorized Owner or Staff member.

Only after approval should payment become available.

The new flow is:

Customer

- Select dates
- Select available bed(s)
- Enter customer details
- Submit Booking Request
- Awaiting Approval

- Owner/Staff reviews request
- Approved
- Payment button becomes enabled
- Customer pays
- Backend verifies payment
- Booking becomes CONFIRMED

BOOKING STATUSES

Use clear states:

- PENDING_APPROVAL
- APPROVED_PAYMENT_PENDING
- CONFIRMED
- CHECKED_IN
- CHECKED_OUT
- REJECTED
- CANCELLED
- EXPIRED

Payment statuses:

- NOT_ENABLED
- PENDING
- SUCCESS
- FAILED
- REFUNDED where supported

Do not confuse booking approval with payment confirmation.

4. CUSTOMER EXPERIENCE BEFORE APPROVAL

After the customer submits a booking request:

Show:

"Booking request submitted."

"Your booking is awaiting approval from Arthayog Dormitory."

Do NOT show an active Pay Now button.

Instead show:

"Payment available after approval."

The customer can still view:

- Booking ID
- Requested dates
- Selected floor
- Selected/assigned bed(s)
- Number of beds
- Booking type
- Requested amount
- Booking status
- Payment status

The customer should clearly understand:

This is a REQUEST, not yet a confirmed booking.

Do not call it "Confirmed" before payment is successfully verified.

=====
5. OWNER/STAFF APPROVAL
=====

In the existing Owner/Staff booking management area, create/use:

"Pending Booking Requests"

For each request show:

- Customer name
- Phone
- Email if available
- Booking ID
- Check-in date
- Check-out date
- Number of beds
- Floor
- Booking type
- Requested amount
- Current availability
- Request date/time

Provide:

[Approve Booking]

[Reject Booking]

Only authorized Owner/Staff users can approve or reject.

Backend MUST enforce authorization.

Do not trust frontend role selection or frontend button visibility.

APPROVAL RESULT

When approved:

PENDING_APPROVAL
→ APPROVED_PAYMENT_PENDING

Payment:

NOT_ENABLED
→ PENDING

Only after the backend records the approved state should the customer see:

[Pay Now]

When rejected:

PENDING_APPROVAL
→ REJECTED

Payment remains disabled.

If the system supports a rejection reason, display it to the customer.

=====
6. CRITICAL PAYMENT SECURITY
=====

The Pay Now button must NOT simply be enabled based on frontend state.

The payment backend/API must verify:

1. Booking exists.
2. Booking belongs to the authenticated customer.
3. Booking status is APPROVED_PAYMENT_PENDING.
4. Payment has not already succeeded.

5. Booking has not expired.
6. Booking has not been cancelled or rejected.
7. Payment amount matches the approved booking amount.
8. Beds/dates are still valid according to the existing availability rules.

If the booking has not been approved, the backend MUST reject any payment attempt.

A customer must not be able to bypass approval by directly calling the payment API.

Never rely only on hiding or disabling the Pay Now button in the frontend.

=====

7. AVAILABILITY RE-CHECK BEFORE PAYMENT

=====

When Owner/Staff approves a booking, re-check current availability.

If the requested beds/dates are no longer available:

- Do not incorrectly approve the booking.
- Inform staff that availability has changed.
- Do not allow double booking.

If the application uses temporary holds for pending requests, use the existing availability/expiration logic.

Pending requests must not permanently block beds.

=====

8. GROUP / WEDDING BOOKING

=====

The approval workflow must also support:

- Individual Booking
- Group Booking
- Wedding Booking
- Multiple-floor booking
- Entire-property booking

For Group/Wedding requests, Staff/Owner should see:

- Selected floor(s)
- Number of beds
- Automatically assigned beds
- Check-in/out dates
- Booking amount

- Availability

Owner/Staff can approve the group booking after reviewing it.

After approval, payment becomes available.

Do not require the customer or staff to manually select every bed when using the existing bulk/group floor booking functionality.

=====

9. CUSTOMER "MY BOOKINGS"

=====

Show clear status information.

PENDING_APPROVAL:

"Awaiting Approval"

"Payment Not Available"

APPROVED_PAYMENT_PENDING:

"Approved — Payment Required"

[Pay Now]

CONFIRMED:

"Confirmed"

"Payment Successful"

REJECTED:

"Booking Request Not Approved"

CANCELLED:

"Booking Cancelled"

EXPIRED:

"Booking Request Expired"

The Pay Now button must ONLY be available when:

Booking status = APPROVED_PAYMENT_PENDING

Never show Pay Now for:

- PENDING_APPROVAL
- REJECTED
- CANCELLED
- EXPIRED
- CONFIRMED
- CHECKED_IN
- CHECKED_OUT

=====

10. PAYMENT CONFIRMATION

=====

After the customer pays:

Backend verifies the actual payment.

Only after successful backend verification:

APPROVED_PAYMENT_PENDING
→ CONFIRMED

Payment:

PENDING
→ SUCCESS

Then show the final booking confirmation.

Do not mark a booking confirmed merely because:

- Customer clicked Pay Now
- Payment page opened
- Customer clicked "I Paid"
- Frontend received a success-looking response

Use the existing real payment verification mechanism.

Never create fake payment IDs or fake payment success.

=====

11. PRICE RULE

=====

Continue using the existing owner-configurable pricing system.

The customer should see the exact amount before payment.

The approved booking amount must be preserved.

If the Owner changes the current price later, an already approved/confirmed booking must not unexpectedly change price.

Example:

Booking approved at ₹500

→ Customer pays ₹500

→ Owner later changes current price to ₹600

→ Existing booking remains ₹500

→ New bookings use ₹600

Use the existing database implementation for this.

=====

12. RECEIPT / INVOICE

=====

The existing Invoice/Receipt functionality must follow the new approval/payment workflow.

Do not generate a "Paid" receipt before verified payment.

Receipt should use the exact confirmed booking amount stored in the booking.

Only show payment SUCCESS/PAID after backend payment verification.

=====

13. SECURITY

=====

Preserve and strengthen existing:

- Authentication
- Argon2 password hashing
- JWT/session security where already implemented
- Backend role-based authorization
- Protected API endpoints
- Input validation
- Payment verification
- Database access controls
- Environment variables for secrets
- No payment secrets in frontend
- No database credentials in frontend
- No frontend-only authorization

- No frontend-only payment confirmation

Customer must only access their own bookings.

Staff must only have the permissions intended for Staff.

Owner has authorized full management access.

=====

14. MULTILINGUAL SUPPORT

=====

Keep:

English
मराठी
हिंदी

The new booking approval/payment messages must also be properly localized.

Do not translate word-for-word if the meaning becomes unnatural.

Use context-aware, natural hospitality terminology.

Examples:

"Awaiting Approval"

"Payment available after approval."

"Booking request submitted."

"Your booking has been approved. Payment is now available."

Provide natural English, Marathi, and Hindi versions.

Do not automatically translate customer names, phone numbers, emails, booking IDs, payment IDs, or user-entered information.

=====

15. UI DESIGN

=====

Maintain the existing Arthayog visual style:

- Warm white
- Clean professional hospitality appearance
- Moderate text sizes
- Clear hierarchy
- Soft earthy/terracotta accents

- Minimal unnecessary decoration
- Responsive on mobile, tablet, and desktop

Do not make the Daily Operations page oversized or cluttered.

Keep the interface practical for front-desk staff.

=====

16. NO FAKE DATA

=====

Strictly do not create fake/random production data for:

- Customers
- Bookings
- Payments
- Revenue
- Beds
- Availability
- Staff
- Phone numbers
- Prices
- Inventory
- Audit logs
- Dashboard statistics

If development test data is required, clearly isolate it from production data.

=====

17. FINAL TESTING

=====

Test this complete customer flow:

Customer submits booking
 → PENDING_APPROVAL
 → Customer cannot pay
 → Staff/Owner sees pending request
 → Staff/Owner approves
 → APPROVED_PAYMENT_PENDING
 → Customer sees Pay Now
 → Customer pays
 → Backend verifies payment
 → CONFIRMED
 → Receipt available

Test rejection:

Customer submits booking
→ PENDING_APPROVAL
→ Staff rejects
→ REJECTED
→ Payment remains disabled

Test security:

Customer attempts payment API before approval
→ Backend rejects request

Test availability:

Two customers attempt to request/book the same bed/date
→ Existing availability rules prevent double booking

Test group booking:

Customer/Staff requests multiple floors
→ Beds are automatically assigned
→ Staff reviews
→ Staff approves
→ Payment becomes available
→ Payment verified
→ Booking confirmed

Test contact:

Owner adds multiple phone numbers
→ Customer sees all active numbers
→ Customer can CALL
→ Inactive/deleted numbers disappear
→ No WhatsApp option appears anywhere

Test Daily Operations:

Staff logs in
→ Daily Operations
→ Sees today's actual arrivals, departures, pending requests and operational actions
→ Can perform required front-desk actions
→ Uses top navigation for detailed Beds, Bookings, or Inventory management.

FINAL PRINCIPLE:

Daily Operations = What staff needs to handle today.

Bookings = Full booking management.

16 Beds = Live bed/floor availability.

Inventory = Inventory management.

Phone Booking = Phone/walk-in booking.

Customer Booking = Booking request first → Owner/Staff approval → Payment → Verified confirmation.

Do not unnecessarily change any other existing working functionality.
Improve the existing ARTHAYOG DORMITORY OWNER/ADMIN interface for a cleaner, more compact and professional look.

IMPORTANT:

- Do NOT rebuild the application.
- Do NOT create a new dashboard.
- Do NOT remove working functionality.
- First inspect the existing Owner dashboard, Real-Time Booking section, Bookings tab, Inventory tab, and Audit Logs.
- Make only UI/layout/content improvements requested below.
- Keep the existing database, APIs, authentication, authorization, booking logic, payment logic, and business logic unchanged unless absolutely required.
- Do not use fake/random data.
- Use the existing dashboard typography and sizing consistently throughout the Owner page.

=====

1. OWNER DASHBOARD TYPOGRAPHY

=====

The current Owner page has some text that is noticeably larger than the rest of the dashboard.

Reduce the oversized text slightly.

Use the SAME typography scale already used in the main Owner dashboard.

Do not introduce large headings or oversized cards.

Maintain a consistent hierarchy:

- Main dashboard headings: moderate size
- Section headings: slightly smaller
- Card titles: compact
- Body text: comfortable but not oversized

- Labels/status text: small and clear
- Buttons: compact and readable

The Owner page should feel dense enough for an operational dashboard without looking crowded.

Do NOT make the interface tiny.

The goal is:
professional + readable + compact + consistent.

=====

2. REAL-TIME BOOKINGS SECTION

=====

The current Real-Time Booking section has text that is too large and creates a cluttered appearance.

Reduce the typography and spacing so that it matches the rest of the Owner dashboard.

Use a cleaner table/list layout.

Do not use oversized cards for every booking.

Show only the most useful information at a glance.

Suggested columns:

- Guest
- Contact
- Check-in
- Check-out
- Booking Type
- Beds
- Floor
- Status
- Payment
- Action

Keep rows compact and aligned.

Use small status badges instead of large blocks of text.

Ensure columns have consistent widths and do not overlap.

On smaller screens, allow proper horizontal scrolling or responsive stacking rather than squeezing columns together.

=====

3. DO NOT DUPLICATE THE OTHER TABS

=====

The Owner dashboard should NOT become a second version of the existing management tabs.

If detailed functionality already exists in:

- Bookings
- Inventory
- 16 Beds
- Reports
- Audit Logs
- Settings

then the Owner dashboard should only show a useful SUMMARY or QUICK STATUS.

Do not reproduce the full content of those tabs on the dashboard.

For example:

Dashboard:

"Today's Bookings: 8"

"Pending Approval: 2"

"Available Beds: 7"

"Revenue Today: ₹XXXX"

Bookings tab:

Full booking management.

Inventory tab:

Full inventory management.

Audit Logs:

Full audit history.

The dashboard should give an overview and allow navigation to the detailed tab when needed.

This will make the Owner page significantly cleaner.

=====

4. BOOKINGS TAB — GUEST & CONTACT

=====

In the Bookings table, the current "Guest & Contact" information appears too continuous/visually merged.

Improve this column.

Instead of displaying everything as one continuous block of text, separate the information visually.

Example:

Guest
Archee Patil

Contact
+91 XXXXX XXXXX

Use a clear hierarchy.

The guest name should be slightly stronger than the contact information.

Keep the contact information smaller and secondary.

Do not make the column unnecessarily wide.

If email is present, display it in a separate secondary line only when useful.

The result should be visually similar to:

Guest Name
Phone number

rather than:

Guest Name • Phone • Email • other information all running continuously.

```
=====
5. BOOKINGS TAB — BED & FLOOR
=====
```

In the Bookings table, there is currently a "Bed & Floor" column.

REMOVE the word/label "Bed" from this entire column.

The column should simply be:

Floor

Show only the floor information.

Examples:

Floor 1
Floor 2
Floor 3
Floor 1 + 2

Do not display:

Bed 1
Bed 2
Bed 3

inside this column.

If exact assigned beds are important for a specific booking, they can remain available inside the booking details page/modal.

The main booking table should stay clean and easy to scan.

=====

6. BOOKINGS TABLE COLUMN ALIGNMENT

=====

Fix the current column layout because the columns look slightly messed up.

Ensure:

- Headers align exactly with their data.
- Column widths are balanced.
- Text does not overlap.
- Long customer names do not break the entire table.
- Status badges stay inside their cells.
- Buttons/actions remain aligned.
- Dates use consistent formatting.
- Currency values align consistently.
- Empty values use a consistent small placeholder such as "—".

Use a clean table structure with appropriate padding.

Do not make every column unnecessarily wide.

Prioritize:

Guest

Contact
Dates
Booking Type
Floor
Status
Payment
Actions

Keep less important information visually secondary.

=====

7. INVENTORY TAB

=====

Review the Inventory interface.

If "Record Usage" is not necessary for the actual small-property workflow, REMOVE the Record Usage action from the main Inventory interface.

Do not create unnecessary inventory operations.

Keep the inventory system focused on:

- Item
- Current quantity
- Low-stock threshold
- Purchase/Add Stock
- Edit
- Remove
- Low-stock status

The Owner should be able to maintain inventory without an unnecessarily complicated workflow.

Do not remove the ability to update stock quantities if that functionality is required.

Simply remove the separate "Record Usage" action if it is redundant.

=====

8. AUDIT LOGS

=====

Remove the phrase:

"Operational Details"

from the Audit Logs interface if it is being used as a heading/category that does not add useful information.

Audit Logs should clearly communicate what happened.

Use a clean structure such as:

User
Role
Action
Affected Record
Timestamp

Where appropriate:

Before
After

Keep the terminology simple.

Do not add unnecessary descriptive labels such as "Operational Details" if they do not provide useful information.

Audit logs must continue using REAL database events only.

Do not generate fake/random audit entries.

=====

9. AUDIT LOG TABLE ALIGNMENT

=====

Fix the Audit Log columns so they are properly aligned.

Use consistent widths and spacing.

Recommended structure:

User | Role | Action | Affected Record | Timestamp

If Before/After information exists, show it in a compact expandable detail view rather than making the main table extremely wide.

The main Audit Log table should remain easy to scan.

Do not make the audit page visually overwhelming.

=====

10. OVERALL OWNER PAGE STYLE

=====

Apply the same visual language throughout the Owner dashboard.

Use:

- Warm white background
- White cards
- Dark readable text
- Muted secondary text
- Subtle borders
- Soft earthy/terracotta accents
- Moderate spacing
- Consistent typography
- Compact operational tables
- Small status badges
- Clean alignment

Avoid:

- Oversized text
- Huge cards
- Excessive empty space
- Excessive shadows
- Large decorative elements
- Repeated navigation cards
- Repeated information
- Unnecessary labels
- Crowded columns
- Long continuous text blocks

=====

11. RESPONSIVE BEHAVIOR

=====

Make sure the improved tables work on:

- Desktop
- Laptop
- Tablet
- Mobile

Do not simply shrink the text until it becomes unreadable.

For tables that cannot reasonably fit on mobile:

Use horizontal scrolling or an appropriate responsive table/card transformation.

Never allow columns to overlap or become visually broken.

=====

12. IMPORTANT FINAL CHECK

=====

After making these changes, inspect the entire Owner dashboard and verify:

- Typography is consistent.
- Real-Time Bookings is compact.
- Dashboard does not duplicate entire tabs.
- Guest & Contact is visually separated.
- "Bed" is removed from the main Floor column.
- Booking columns are aligned.
- Inventory does not show unnecessary "Record Usage".
- "Operational Details" is removed from Audit Logs.
- Audit Log columns are aligned.
- No fake/random data was introduced.
- No existing working backend functionality was broken.
- No authentication or authorization changes were introduced.
- No booking/payment logic was changed unnecessarily.

The final Owner interface should look like a clean professional operations dashboard rather than a collection of large cards and repeated sections.

Update the existing Arthayog Dormitory application by adding a proper:

“Aadhaar Authentication + Live Guest Photo Verification”

feature.

IMPORTANT:

Do NOT rebuild the application.

Do NOT change the existing booking, payment, pricing, beds, roles, dashboards, or other working functionality except where necessary to integrate this feature.

The purpose is:

1. Aadhaar = identity authentication/verification
2. Live photo = verification of the actual person present at check-in

These are TWO SEPARATE processes.

=====

1. AADHAAR AUTHENTICATION

=====

Add Aadhaar identity verification using ONLY a proper authorized/UIDAI-compliant Aadhaar authentication or offline-verification mechanism.

Do NOT build a custom Aadhaar facial-recognition system.

Do NOT compare the live guest photograph with the Aadhaar photograph.

Do NOT store Aadhaar biometric information.

Do NOT permanently store Aadhaar authentication data that is prohibited from being retained.

Do NOT expose Aadhaar details to other customers or unauthorized staff.

If no authorized Aadhaar verification provider/API is configured yet, DO NOT create fake verification.

Instead show:

“Aadhaar verification service is not configured.”

The system must clearly distinguish between:

- Aadhaar verification
- Live guest photo verification

=====

2. AADHAAR VERIFICATION FLOW

=====

For a booking/check-in:

Guest/Staff starts identity verification.

Collect only the minimum Aadhaar information required by the configured authorized verification method.

Show appropriate consent/information before verification.

After successful authorized verification, store only the minimum legally permitted verification information, such as:

- Verification status: VERIFIED / FAILED / NOT_VERIFIED
- Verification date and time
- Verification method
- Provider/reference ID only if legally permitted

- Related guest/booking ID

Do NOT store unnecessary Aadhaar information.

Do NOT display the full Aadhaar number in normal booking screens.

If an Aadhaar number must be displayed anywhere, mask it.

Example:

XXXX-XXXX-1234

Never store Aadhaar passwords, OTPs, biometric data, or authentication secrets.

=====

3. LIVE GUEST PHOTO

=====

Add a separate live camera photo feature.

At check-in, Owner/Staff should have a button:

“Capture Guest Photo”

When clicked:

- Open the device camera
- Allow the staff to capture the actual guest's photograph
- Show a preview before saving
- Allow Retake
- Allow Confirm Photo
- Save the confirmed photo securely
- Link the photo to the guest record and relevant booking/check-in record

The live photo is NOT Aadhaar biometric data.

The live photo must NOT be automatically compared with the Aadhaar photograph.

The system must NOT perform face recognition or 1:N facial searching.

The purpose is simply:

“Record and verify the person physically present at check-in.”

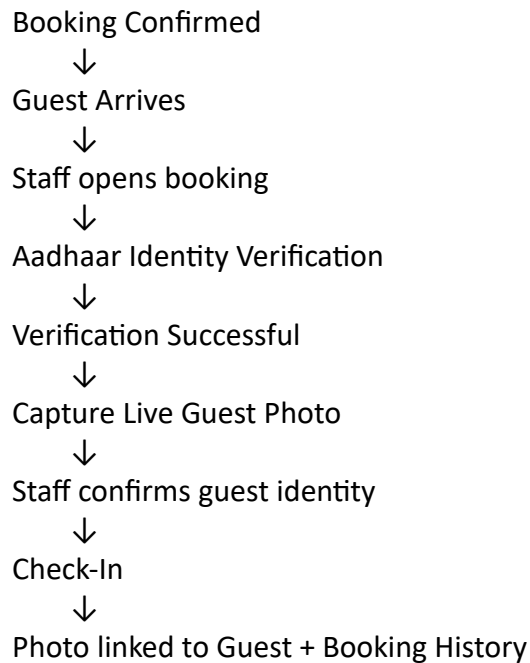
=====

4. CHECK-IN WORKFLOW

=====

Integrate this into the existing check-in process.

Recommended flow:



Do not automatically mark check-in complete if required verification steps have failed.

If Aadhaar verification fails:

Show:

“Identity verification could not be completed.”

Allow Staff/Owner to retry or follow the configured alternative identification procedure.

Do NOT fake a successful verification.

=====

5. OWNER/STAFF BOOKING SCREEN

=====

Inside Booking Details, add an:

“Identity Verification”

section.

Display:

Identity Verification

Aadhaar Verification: VERIFIED

Verified On: DD/MM/YYYY

Verification Method: [configured method]

Guest Photo: AVAILABLE

Captured On: DD/MM/YYYY HH:MM

Buttons:

[Verify Identity]

[Capture Guest Photo]

[View Guest Photo]

[Retake Photo]

Only authorized Owner/Staff roles can access these functions.

Customers must NOT be able to view another guest's identity information or live photo.

=====

6. GUEST PROFILE / HISTORY

=====

Extend the existing guest history.

For each guest, show:

Guest Information

- Name
- Phone
- Email where available
- Booking history

Identity Information

- Aadhaar Verification Status
- Verification Date
- Verification Method

Guest Photo

- Latest permitted live guest photo
- Photo captured date/time

Booking History

- Booking ID
- Check-in
- Check-out

- Floor
- Assigned beds
- Booking status

Do not display unnecessary Aadhaar information.

Do not create duplicate guest profiles when the same verified guest books again.

Where appropriate, allow authorized staff to associate a new booking with an existing guest record.

=====

7. PHOTO SECURITY

=====

Store guest photos securely using the application's existing secure storage/backend.

Do NOT store photos as publicly accessible URLs.

Do NOT put photos directly into frontend source code.

Do NOT expose photo storage publicly.

Only authorized Owner/Staff should be able to retrieve guest photos.

Use authenticated backend access to retrieve the photo.

Do not put sensitive photo information into logs.

Add a configurable retention/deletion policy for guest photos.

The Owner should be able to configure how long guest photos are retained where appropriate.

After the retention period, the system should support secure deletion.

=====

8. DATABASE

=====

Use the existing database.

Do not create an unnecessary separate database.

Add appropriate fields/tables/relationships.

Suggested structure:

Guest

- id
- name
- phone
- email
- address where required
- identity_verification_status
- identity_verified_at
- identity_verification_method
- identity_verification_reference
- created_at
- updated_at

GuestPhoto

- id
- guest_id
- booking_id
- secure_storage_reference
- captured_at
- captured_by_user_id
- status
- retention/deletion information
- created_at

IdentityVerification

- id
- guest_id
- booking_id
- status
- verification_method
- provider_reference
- verified_at
- verified_by
- created_at

Do NOT store prohibited Aadhaar biometric/authentication information.

Do not duplicate information unnecessarily.

=====

9. ROLE PERMISSIONS

=====

Follow the existing Arthayog role system:

OWNER / ADMIN:

- Full access
- Can verify identity
- Can capture/view guest photos
- Can manage retention/settings

STAFF:

- Can perform identity verification
- Can capture guest photo
- Can view photos for operational purposes
- Cannot change security configuration unless already authorized

CUSTOMER:

- Cannot access other guests' identity information
- Cannot access other guests' photos
- Can only see their own applicable booking/verification status

Enforce these permissions in the BACKEND.

Do NOT rely only on hiding buttons in React.

=====

10. AUDIT LOG

=====

Use the existing Audit Logs system.

Record important actions such as:

- Identity verification attempted
- Identity verification successful
- Identity verification failed
- Guest photo captured
- Guest photo replaced
- Guest photo deleted
- Verification information updated

Audit log should contain:

User
 Role
 Action
 Affected Guest/Booking
 Timestamp

Do NOT put Aadhaar numbers, OTPs, biometric information, or sensitive authentication data into audit logs.

=====

11. CUSTOMER UI

=====

Do not unnecessarily expose sensitive identity information to customers.

If the customer needs to know the verification state, show something simple:

Identity Verification

“Verification completed”

or

“Identity verification pending”

Do not show internal verification references or sensitive Aadhaar information.

=====

12. ERROR HANDLING

=====

Handle:

- Camera permission denied
- Camera unavailable
- Photo capture failed
- Upload failed
- Aadhaar verification failed
- Aadhaar service unavailable
- Verification timeout
- Network failure
- Unauthorized access
- Expired session
- Invalid booking
- Guest record not found

Use clear messages.

Never show technical stack traces to the customer.

Never mark identity as VERIFIED when verification has not actually succeeded.

=====

13. IMPORTANT PRIVACY RULES

=====

Follow these rules throughout the implementation:

- Aadhaar authentication and live photo verification are separate.
- No custom Aadhaar face matching.
- No face-recognition database.
- No 1:N face search.
- No storage of Aadhaar biometrics.
- No storage of OTP/authentication secrets.
- Store only legally permitted verification information.
- Obtain appropriate consent/information where required.
- Secure guest photos.
- Restrict access using backend authorization.
- Keep an audit trail without sensitive Aadhaar data.
- Provide retention/deletion controls for guest photos.
- Never create fake Aadhaar verification.

=====

14. UI DESIGN

=====

Match the existing Arthayog design.

Use the existing:

- Warm white background
- White cards
- Dark text
- Neutral borders
- Professional hospitality style
- Moderate typography
- Existing dashboard typography

Do NOT create a new blue/purple SaaS-style interface.

Keep the identity section compact and professional.

Use clear status badges:

VERIFIED
PENDING
FAILED
NOT VERIFIED

Do not make the identity section unnecessarily large.

=====

15. FINAL INTEGRATION

=====

Integrate this into the EXISTING Arthayog workflow:

Customer Booking

→ Booking Approval

→ Payment

→ Confirmed Booking

→ Guest Arrival

→ Identity Verification

→ Live Guest Photo

→ Check-In

→ Guest History

→ Check-Out

Do not change the existing approval-before-payment workflow.

Do not change existing bed availability logic.

Do not change existing pricing logic.

Do not change existing payment verification.

Do not change Owner/Staff/Customer authentication.

Only extend the existing system with the identity verification and guest photo functionality.

=====

16. TEST EVERYTHING

=====

After implementation, test:

1. Authorized Aadhaar verification succeeds.
2. Failed Aadhaar verification does not become VERIFIED.
3. No fake Aadhaar verification is generated.
4. Camera opens correctly.
5. Guest photo can be captured.
6. Photo can be retaken.
7. Photo is securely stored.
8. Photo is linked to the correct guest.
9. Photo is linked to the correct booking.
10. Owner can view it.
11. Authorized Staff can view it.
12. Customer cannot view another guest's photo.
13. Unauthorized API requests are rejected.
14. Aadhaar sensitive information is not exposed.

15. Audit logs contain no sensitive Aadhaar data.
16. Guest history displays previous permitted verification/photo information correctly.
17. Check-in cannot incorrectly show successful verification when verification failed.
18. Existing booking/payment/bed functionality continues working.

IMPORTANT FINAL REQUIREMENT:

Do not implement a fake Aadhaar API, fake verification result, fake camera result, or placeholder “verified” status.

If an authorized Aadhaar integration is not currently available in the project, build the complete UI, database structure, permission model, and integration interface so the real authorized provider can be connected later, but clearly show:

“Aadhaar verification service is not configured.”

Never pretend that verification has occurred when it has not.